

Dos and Don'ts of Machine Learning in Computer Security

Daniel Arp^{*}, Erwin Quiring[†], Feargus Pendlebury^{‡§}, Alexander Warnecke[†], Fabio Pierazzi[‡],
Christian Wressnegger[¶], Lorenzo Cavallaro^{||}, Konrad Rieck[†]

^{*}*Technische Universität Berlin*

[†]*Technische Universität Braunschweig*

[‡]*King's College London*, ^{||}*University College London*

[§]*Royal Holloway, University of London and The Alan Turing Institute*

[¶]*KASTEL Security Research Labs and Karlsruhe Institute of Technology*

Abstract

With the growing processing power of computing systems and the increasing availability of massive datasets, machine learning algorithms have led to major breakthroughs in many different areas. This development has influenced computer security, spawning a series of work on learning-based security systems, such as for malware detection, vulnerability discovery, and binary code analysis. Despite great potential, machine learning in security is prone to subtle pitfalls that undermine its performance and render learning-based systems potentially unsuitable for security tasks and practical deployment.

In this paper, we look at this problem with critical eyes. First, we identify common pitfalls in the design, implementation, and evaluation of learning-based security systems. We conduct a study of 30 papers from top-tier security conferences within the past 10 years, confirming that these pitfalls are widespread in the current security literature. In an empirical analysis, we further demonstrate how individual pitfalls can lead to unrealistic performance and interpretations, obstructing the understanding of the security problem at hand. As a remedy, we propose actionable recommendations to support researchers in avoiding or mitigating the pitfalls where possible. Furthermore, we identify open problems when applying machine learning in security and provide directions for further research.

1 Introduction

No day goes by without reading machine learning success stories. The widespread access to specialized computational resources and large datasets, along with novel concepts and architectures for deep learning, have paved the way for machine learning breakthroughs in several areas, such as the translation of natural languages [13, 31, 125] and the recognition of image content [62, 78, 117]. This development has naturally influenced security research: although mostly confined to specific applications in the past [53, 54, 132], machine learning has now become one of the key enablers to studying

and addressing security-relevant problems at large in several application domains, including intrusion detection [43, 93], malware analysis [69, 88], vulnerability discovery [83, 142], and binary code analysis [42, 114, 140].

Machine learning, however, has no clairvoyant abilities and requires reasoning about statistical properties of data across a fairly delicate workflow: incorrect assumptions and experimental biases may cast doubts on this process to the extent that it becomes unclear whether we can trust scientific discoveries made using learning algorithms at all [56]. Attempts to identify such challenges and limitations in specific security domains, such as network intrusion detection, started two decades ago [11, 119, 126] and were extended more recently to other domains, such as malware analysis and website fingerprinting [3, 72, 104, 112]. Orthogonal to this line of work, however, we argue that there exist *generic pitfalls* related to machine learning that affect all security domains and have received little attention so far.

These pitfalls can lead to over-optimistic results and, even worse, affect the entire machine learning workflow, weakening assumptions, conclusions, and lessons learned. As a consequence, a false sense of achievement is felt that hinders the adoption of research advances in academia and industry. A sound scientific methodology is fundamental to support intuitions and draw conclusions. We argue that this need is especially relevant in security, where processes are often undermined by adversaries that actively aim to bypass analysis and break systems.

In this paper, we identify ten common—yet subtle—pitfalls that pose a threat to validity and hinder interpretation of research results. To support this claim, we analyze the prevalence of these pitfalls in 30 top-tier security papers from the past decade that rely on machine learning for tackling different problems. To our surprise, each paper suffers from at least three pitfalls; even worse, several pitfalls affect most of the papers, which shows how endemic and subtle the problem is. Although the pitfalls are widespread, it is perhaps more important to understand the extent to which they weaken results and lead to over-optimistic conclusions. To this end,

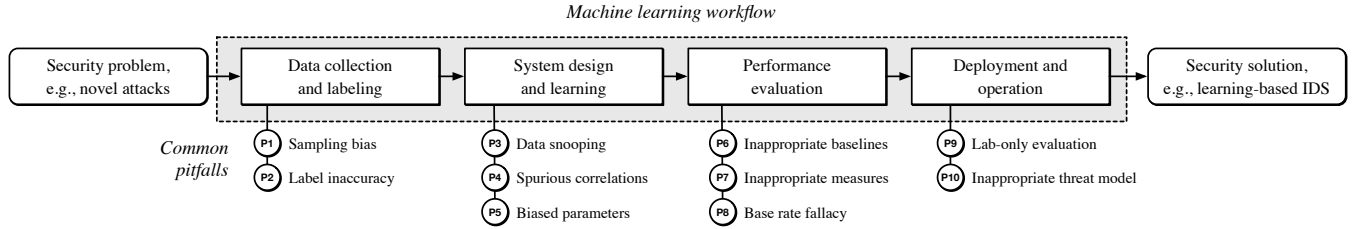


Figure 1: Common pitfalls of machine learning in computer security.

we perform an impact analysis of the pitfalls in four different security fields. The findings support our premise echoing the broader concerns of the community.

In summary, we make the following contributions:

1. **Pitfall Identification.** We identify ten pitfalls as *don'ts* of machine learning in security and propose *dos* as actionable recommendations to support researchers in avoiding the pitfalls where possible. Furthermore, we identify open problems that cannot be mitigated easily and require further research effort (§2).
2. **Prevalence Analysis.** We analyze the prevalence of the identified pitfalls in 30 representative top-tier security papers published in the past decade. Additionally, we perform a broad survey in which we obtain and evaluate the feedback of the authors of these papers regarding the identified pitfalls (§3).
3. **Impact Analysis.** In four different security domains, we experimentally analyze the extent to which such pitfalls introduce experimental bias, and how we can effectively overcome these problems by applying the proposed recommendations (§4).

Remark. This work should not be interpreted as a finger-pointing exercise. On the contrary, it is a reflective effort that shows how subtle pitfalls can have a negative impact on progress of security research, and how we—as a community—can mitigate them adequately.

2 Pitfalls in Machine Learning

Despite its great success, the application of machine learning in practice is often non-trivial and prone to several pitfalls, ranging from obvious flaws to minor blemishes. Overlooking these issues may result in experimental bias or incorrect conclusions, especially in computer security. In this section, we present ten common pitfalls that occur frequently in security research. Although some of these pitfalls may seem obvious at first glance, they are rooted in subtle deficiencies that are widespread in security research—even in papers presented at top conferences (see §3 and §4).

We group these pitfalls with respect to the stages of a typical machine learning workflow, as depicted in Figure 1. For each pitfall, we provide a short description, discuss its impact on the security domain, and present recommendations. Moreover, a colored bar depicts the proportion of papers in our analysis that suffer from the pitfall, with warmer colors indicating the presence of the pitfall (see Figure 3).

2.1 Data Collection and Labeling

The design and development of learning-based systems usually starts with the acquisition of a representative dataset. It is clear that conducting experiments using unrealistic data leads to the misestimation of an approach’s capabilities. The following two pitfalls frequently induce this problem and thus require special attention when developing learning-based systems in computer security.

P1 – Sampling Bias. The collected data does not sufficiently represent the true data distribution of the underlying security problem [1, 30, 33].

60% present

Description. With a few rare exceptions, researchers develop learning-based approaches without exact knowledge of the true underlying distribution of the input space. Instead, they need to rely on a dataset containing a fixed number of samples that aim to resemble the actual distribution. While it is inevitable that some bias exists in most cases, understanding the specific bias inherent to a particular problem is crucial to limiting its impact in practice. Drawing meaningful conclusions from the training data becomes challenging, if the data does not effectively represent the input space or even follows a different distribution.

Security implications. Sampling bias is highly relevant to security, as the acquisition of data is particularly challenging and often requires using multiple sources of varying quality. As an example, for the collection of suitable datasets for Android malware detection only a few public sources exist from which to obtain such data [6, 134]. As a result, it is common practice to rely on synthetic data or to combine data from different sources, both of which can introduce bias as we demonstrate in §4 with examples on state-of-the-art methods for intrusion and malware detection.

Recommendations. In many security applications, sampling from the true distribution is extremely difficult and sometimes even impossible. Consequently, this bias can often only be mitigated but not entirely removed. In §4, we show that, in some cases, a reasonable strategy is to construct different estimates of the true distribution and analyze them individually. Further strategies include the extension of the dataset with synthetic data [e.g., 28, 60, 137] or the use of transfer learning [see 99, 135, 145, 147]. However, the mixing of data from incompatible sources should be avoided, as it is a common cause of additional bias. In any case, limitations of the used dataset should be openly discussed, allowing other researchers to better understand the security implications of potential sampling bias.

P2 – Label Inaccuracy. The ground-truth labels required for classification tasks are inaccurate, unstable, or erroneous, affecting the overall performance of a learning-based system [85, 144].

Description. Many learning-based security systems are built for classification tasks. To train these systems, a ground-truth label is required for each observation. Unfortunately, this labeling is rarely perfect and researchers must account for uncertainty and noise to prevent their models from suffering from inherent bias.

Security implications. For many relevant security problems, such as detecting network attacks or malware, reliable labels are typically not available, resulting in a chicken-and-egg problem. As a remedy, researchers often resort to heuristics, such as using external sources that do not provide a reliable ground-truth. For example, services like *VirusTotal* are commonly used for acquiring label information for malware but these are not always consistent [144]. Additionally, changes in adversary behavior may alter the ratio between different classes over time [3, 92, 144], introducing a bias known as *label shift* [85]. A system that cannot adapt to these changes will experience performance decay once deployed.

Recommendations. Generally, labels should be verified whenever possible, for instance, by manually investigating false positives or a random sample [e.g., 122]. If *noisy labels* cannot be ruled out, their impact on the learning model can be reduced by (i) using robust models or loss functions, (ii) actively modeling label noise in the learning process, or (iii) cleansing noisy labels in the training data [see 55, 67, 84]. To demonstrate the applicability of such approaches, we empirically apply a cleansing approach in Appendix A. Note that instances with uncertain labels must not be removed from the test data. This represents a variation of sampling bias (P1) and data snooping (P3), a pitfall we discuss in detail in §2.2. Furthermore, as labels may change over time, it is necessary to take precautions against *label shift* [85], such as by delaying labeling until a stable ground-truth is available [see 144].

2.2 System Design and Learning

Once enough data has been collected, a learning-based security system can be trained. This process ranges from data preprocessing to extracting meaningful features and building an effective learning model. Unfortunately, flaws and weak spots can be introduced at each of these steps.

P3 – Data Snooping. A learning model is trained with data that is typically not available in practice. Data snooping can occur in many ways, some of which are very subtle and hard to identify [1].

Description. It is common practice to split collected data into separate training and test sets prior to generating a learning model. Although splitting the data seems straightforward, there are many subtle ways in which test data or other background information that is not usually available can affect the training process, leading to data snooping. While a detailed list of data snooping examples is provided in the appendix (see Table 8), we broadly distinguish between three types of data snooping: *test*, *temporal*, and *selective snooping*.

Test snooping occurs when the test set is used for experiments before the final evaluation. This includes preparatory work to identify useful features, parameters, and learning algorithms. Temporal snooping occurs if time dependencies within the data are ignored. This is a common pitfall, as the underlying distributions in many security-related problems are under continuous change [e.g., 87, 104]. Finally, selective snooping describes the cleansing of data based on information not available in practice. An example is the removal of outliers based on statistics of the complete dataset (i.e., training and test) that are usually not available at training time.

Security implications. In security, data distributions are often non-stationary and continuously changing due to new attacks or technologies. Because of this, snooping on data from the future or from external data sources is a prevalent pitfall that leads to over-optimistic results. For instance, several researchers have identified temporal snooping in learning-based malware detection systems [e.g., 4, 8, 104]. In all these cases, the capabilities of the methods are overestimated due to mixing samples from past and present. Similarly, there are incidents of test and selective snooping in security research that lead to unintentionally biased results (see §3).

Recommendations. While it seems obvious that training, validation, and test data should be strictly separated, this data isolation is often unintentionally violated during the preprocessing stages. For example, we observe that it is a common mistake to compute tf-idf weights or neural embeddings over the entire dataset (see §3). To avoid this problem, test data should be split early during data collection and stored separately until the final evaluation. Furthermore, temporal dependencies within the data should be considered when creating

the dataset splits [4, 87, 104]. Other types of data snooping, however, are challenging to address. For instance, as the characteristics of publicly available datasets are increasingly exposed, methods developed using this data implicitly leverage knowledge from the test data [see 1, 90]. Consequently, experiments on well-known datasets should be complemented with experiments on more recent data from the considered application domain.

P4 – Spurious Correlations. Artifacts unrelated to the security problem create shortcut patterns for separating classes. Consequently, the learning model adapts to these artifacts instead of solving the actual task.

Description. Spurious correlations result from artifacts that correlate with the task to solve but are not actually related to it, leading to false associations. Consider the example of a network intrusion detection system, where a large fraction of the attacks in the dataset originate from a certain network region. The model may learn to detect a specific IP range instead of generic attack patterns. Note that while sampling bias is a common reason for spurious correlations, these can also result from other factors, as we discuss in more detail in Appendix A.

Security implications. Machine learning is typically applied as a black box in security. As a result, spurious correlations often remain unidentified. These correlations pose a problem once results are interpreted and used for drawing general conclusions. Without knowledge of spurious correlations, there is a high risk of overestimating the capabilities of an approach and misjudging its practical limitations. As an example, §4.2 reports our analysis on a vulnerability discovery system indicating the presence of notable spurious correlations in the underlying data.

Recommendations. To gain a better view of the capabilities of a learning-based systems, we generally recommend applying explanation techniques for machine learning [see 59, 79, 133]. Despite some limitations [e.g., 66, 75, 127], these techniques can reveal spurious correlations and allow a practitioner to assess their impact on the system’s capabilities. As an example, we show for different security-related problems how explainable learning can help to identify this issue in §4. Note that spurious correlations in one setting may be considered a valid signal in another, depending on the objective of the learning-based system. Consequently, we recommend clearly defining this objective in advance and validating whether correlations learned by the system comply with this goal. For example, a robust malware detection system should pick up features related to malicious activity rather than other unrelated information present in the data.

P5 – Biased Parameter Selection. The final parameters of a learning-based method are not entirely fixed at training time. Instead, they indirectly depend on the test set.

Description. Throughout the learning procedure, it is common practice to generate different models by varying hyperparameters. The best-performing model is picked and its performance on the test set is presented. While this setup is generally sound, it can still suffer from a biased parameter selection. For example, over-optimistic results can be easily produced by tuning hyperparameters or calibrating thresholds on the test data instead of the training data.

Security implications. A security system whose parameters have not been fully calibrated at training time can perform very differently in a realistic setting. While the detection threshold for a network intrusion detection system may be chosen using a ROC curve obtained on the test set, it can be hard to select the same operational point in practice due the diversity of real-world traffic [119]. This may lead to decreased performance of the system in comparison to the original experimental setting. Note that this pitfall is related to data snooping (P3), but should be considered explicitly as it can easily lead to inflated results.

Recommendations. This pitfall constitutes a special case of data snooping and thus the same countermeasures apply. However, in practice fixing a biased parameter selection can often be easily achieved by using a separate *validation set* for model selection and parameter tuning. In contrast to general data snooping, which is often challenging to mitigate, strict data isolation is already sufficient to rule out problems when determining hyperparameters and thresholds.

2.3 Performance Evaluation

The next stage in a typical machine-learning workflow is the evaluation of the system’s performance. In the following, we show how different pitfalls can lead to unfair comparisons and biased results in the evaluation of such systems.

P6 – Inappropriate Baseline. The evaluation is conducted without, or with limited, baseline methods. As a result, it is impossible to demonstrate improvements against the state of the art and other security mechanisms.

Description. To show to what extent a novel method improves the state of the art, it is vital to compare it with previously proposed methods. When choosing baselines, it is important to remember that there exists no universal learning algorithm that outperforms all other approaches in general [136]. Consequently, providing only results for the proposed approach or a comparison with mostly identical learning models, does not give enough context to assess its impact.

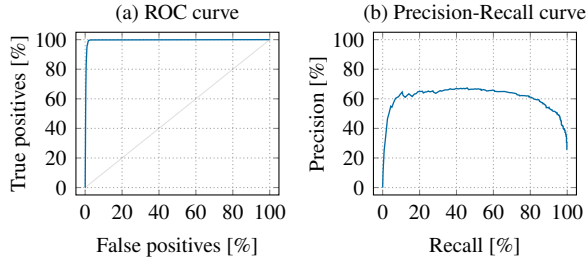


Figure 2: ROC and precision-recall curve as two performance measures for the same scores, created on an artificial dataset with an imbalanced class ratio. Only the precision-recall curve conveys the true performance.

Security implications. An overly complex learning method increases the chances of overfitting, and also the runtime overhead, the attack surface, and the time and costs for deployment. To show that machine learning techniques provide significant improvements compared to traditional methods, it is thus essential to compare these systems side by side.

Recommendations. Instead of focusing solely on complex models for comparison, simple models should also be considered throughout the evaluation. These methods are easier to explain, less computationally demanding, and have proven to be effective and scalable in practice. In §4, we demonstrate how using well-understood, simple models as a baseline can expose unnecessarily complex learning models. Similarly, we show that automated machine learning (*AutoML*) frameworks [e.g., 48, 70] can help finding proper baselines. While these automated methods can certainly not replace experienced data analysts, they can be used to set the lower bar the proposed approach should aim for. Finally, it is critical to check whether non-learning approaches are also suitable for the application scenario. For example, for intrusion and malware detection, there exist a wide range of methods using other detection strategies [e.g., 45, 102, 111].

P7 – Inappropriate Performance Measures. The chosen performance measures do not account for the constraints of the application scenario, such as imbalanced data or the need to keep a low false-positive rate.

Description. A wide range of performance measures are available and not all of them are suitable in the context of security. For example, when evaluating a detection system, it is typically insufficient to report just a single performance value, such as the accuracy, because true-positive and false-positive decisions are not observable. However, even more advanced measures, such as ROC curves, may obscure experimental results in some application settings. Figure 2 shows an ROC curve and a precision-recall curve on an imbalanced dataset (class ratio 1:100). Given the ROC curve alone, the performance appears excellent, yet the low precision reveals the true performance of the classifier, which would be impractical for many security applications.

Furthermore, various security-related problems deal with more than two classes, requiring *multi-class metrics*. This setting can introduce further subtle pitfalls. Common strategies, such as *macro-averaging* or *micro-averaging* are known to overestimate and underestimate small classes [51].

Security implications. Inappropriate performance measures are a long-standing problem in security research, particularly in detection tasks. While true and false positives, for instance, provide a more detailed picture of a system’s performance, they can also disguise the actual precision when the prevalence of attacks is low.

Recommendations. The choice of performance measures in machine learning is highly application-specific. Hence, we refrain from providing general guidelines. Instead, we recommend considering the practical deployment of a learning-based system and identifying measures that help a practitioner assess its performance. Note that these measures typically differ from standard metrics, such as the accuracy or error, by being more aligned with day-to-day operation of the system. To give the reader an intuition, in §4.1, we show how different performance measures for an Android malware detector lead to contradicting interpretations of its performance.

P8 – Base Rate Fallacy. A large class imbalance is ignored when interpreting the performance measures leading to an overestimation of performance.

Description. Class imbalance can easily lead to a misinterpretation of performance if the base rate of the negative class is not considered. If this class is predominant, even a very low false-positive rate can result in surprisingly high numbers of false positives. Note the difference to the previous pitfall: while P7 refers to the inappropriate *description* of performance, the base-rate fallacy is about the misleading *interpretation* of results. This special case is easily overlooked in practice (see §3). Consider the example in Figure 2 where 99 % true positives are possible at 1 % false positives. Yet, if we consider the class ratio of 1:100, this actually corresponds to 100 false positives for every 99 true positives.

Security implications. The base rate fallacy is relevant in a variety of security problems, such as intrusion detection and website fingerprinting [e.g., 11, 72, 100]. As a result, it is challenging to realistically quantify the security and privacy threat posed by attackers. Similarly, the probability of installing malware is usually much lower than is considered in experiments on malware detection [104].

Recommendations. Several problems in security revolve around detecting rare events, such as threats and attacks. For these problems, we advocate the use of *precision* and *recall* as well as related measures, such as precision-recall curves. In contrast to other measures, these functions account for class imbalance and thus resemble reliable performance indicators

for detection tasks focusing on a minority class [38, 118]. However, note that precision and recall can be misleading if the prevalence of the minority class is inflated, for example, due to sampling bias [104]. In these cases, other measures like *Matthews Correlation Coefficient (MCC)* are more suitable to assess the classifier’s performance [29] (see §4). In addition, ROC curves and their AUC values are useful measures for comparing detection and classification approaches. To put more focus on practical constraints, we recommend considering the curves only up to tractable false-positive rates and to compute bounded AUC values. Finally, we also recommend discussing false positives in relation to the base rate of the negative class, which enables the reader to get an understanding of the workload induced by false-positive decisions.

2.4 Deployment and Operation

In the last stage of a typical machine-learning workflow, the developed system is deployed to tackle the underlying security problem in practice.

P9 – Lab-Only Evaluation. A learning-based system is solely evaluated in a laboratory setting, without discussing its practical limitations.

47% present

Description. As in all empirical disciplines, it is common to perform experiments under certain assumptions to demonstrate a method’s efficacy. While performing controlled experiments is a legitimate way to examine specific aspects of an approach, it should be evaluated in a realistic setting whenever possible to transparently assess its capabilities and showcase the open challenges that will foster further research.

Security implications. Many learning-based systems in security are evaluated solely in laboratory settings, overstating their practical impact. A common example are detection methods evaluated only in a *closed-world setting* with limited diversity and no consideration of non-stationarity [15, 71]. For example, a large number of website fingerprinting attacks are evaluated only in closed-world settings spanning a limited time period [72]. Similarly, several learning-based malware detection systems have been insufficiently examined under realistic settings [see 5, 104].

Recommendations. It is essential to move away from a *laboratory setting* and approximate a *real-world setting* as accurately as possible. For example, temporal and spatial relations of the data should be considered to account for the typical dynamics encountered in the wild [see 104]. Similarly, runtime and storage constraints should be analyzed under practical conditions [see 15, 112, 130]. Ideally, the proposed system should be deployed to uncover problems that are not observable in a lab-only environment, such as the diversity of real-world network traffic [see 119]—although this is not always possible due to ethical and privacy constraints.

P10 – Inappropriate Threat Model. The security of machine learning is not considered, exposing the system to a variety of attacks, such as poisoning and evasion attacks.

17% present

Description. Learning-based security systems operate in a hostile environment, which should be considered when designing these systems. Prior work in adversarial learning has revealed a considerable attack surface introduced by machine learning itself, at all stages of the workflow [see 18, 101]. Their broad attack surface makes these algorithms vulnerable to various types of attacks, such as adversarial preprocessing, poisoning, and evasion [e.g., 19, 20, 25, 105, 108].

Security implications. Including adversarial influence in the threat model and evaluation is often vital, as systems prone to attacks are not guaranteed to output trustworthy and meaningful results. Aside from traditional security issues, it is therefore essential to also consider machine learning-related attacks. For instance, an attacker may more easily evade a model that relies on only a few features than a properly regularized model that has been designed with security considerations in mind [40], although one should also consider domain-specific implications [105]. Furthermore, *semantic gaps* in the workflow of machine learning may create blind spots for attacks. For example, imprecise parsing and feature extraction may enable an adversary to hide malicious content [131].

Recommendations. In most fields of security where learning-based systems are used, we operate in an *adversarial environment*. Hence, threat models should be defined precisely and systems evaluated with respect to them. In most cases, it is necessary to assume an *adaptive adversary* that specifically targets the proposed systems and will search for and exploit weaknesses for evasion or manipulation. Similarly, it is crucial to consider all stages of the machine learning workflow and investigate possible vulnerabilities [see 18, 26, 39, 101]. For this analysis, we recommend focusing on white-box attacks where possible, following Kerckhoff’s principle [73] and security best practices. Ultimately, we like to stress that an evaluation of adversarial aspects is not an add-on but rather a mandatory component in security research.

3 Prevalence Analysis

Once we understand the pitfalls faced by learning-based security systems, it becomes necessary to assess their prevalence and investigate their impact on scientific advances. To this end, we conduct a study on 30 papers published in the last ten years at ACM CCS, IEEE S&P, USENIX Security, and NDSS, the top-4 conferences for security-related research in our community. The papers have been selected as representative examples for our study, as they address a large variety of security topics and successfully apply machine learning to the corresponding research problems.

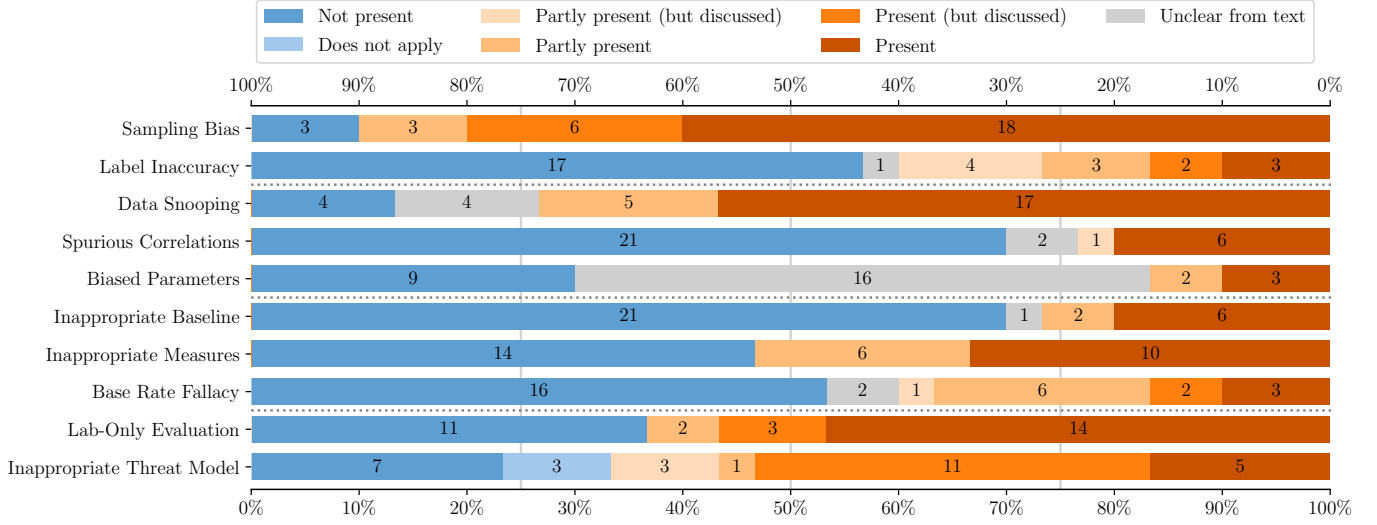


Figure 3: Stacked bar chart showing the pitfalls suffered by each of the 30 papers analyzed. The colors of each bar show the degree to which a pitfall was present, and the width shows the proportion of papers in that group. The number at the center of each bar shows the cardinality of each group.

In particular, our selection of top-tier papers covers the following topics: malware detection [9, 34, 88, 104, 121, 138]; network intrusion detection [43, 93, 113, 115]; vulnerability discovery [42, 49, 50, 83]; website fingerprint attacks [44, 100, 110, 116]; social network abuse [22, 95, 120]; binary code analysis [14, 32, 114]; code attribution [2, 23]; steganography [17]; online scams [74]; game bots [80]; and ad blocking [68].

Review process. Each paper is assigned two independent reviewers who assess the article and identify instances of the described pitfalls. The pool of reviewers consists of six researchers who have all previously published work on the topic of machine learning and security in at least one of the considered security conferences. Reviewers do *not* consider any material presented outside the papers under analysis (aside from appendices and associated artifacts, such as datasets or source code). Once both reviewers have completed their assignments, they discuss the paper in the presence of a third reviewer that may resolve any disputes. In case of uncertainty, the authors are given the benefit of the doubt (e.g., in case of a dispute between *partly present* and *present*, we assign *partly present*).

Throughout the process, all reviewers meet regularly in order to discuss their findings and ensure consistency between the pitfalls’ criteria. Moreover, these meetings have been used to refine the definitions and scope of pitfalls based on the reviewers’ experience. Following any adaptation of the criteria, all completed reviews have been re-evaluated by the original reviewers—this occurred twice during our analysis. While cumbersome, this adaptive process of incorporating reviewer feedback ensures that the pitfalls are comprehensive in describing the core issues across the state of the art. We note that the inter-rater reliability of reviews prior to dispute resolution is $\alpha = 0.832$ using Krippendorff’s alpha, where $\alpha > 0.800$ indicates confidently reliable ratings [77].

Assessment criteria. For each paper, pitfalls are coarsely classified as either *present*, *not present*, *unclear from text*, or *does not apply*. A pitfall may be wholly present throughout the experiments without remediation (*present*), or it may not (*not present*). If the authors have corrected any bias or have narrowed down their claims to accommodate the pitfall, this is also counted as *not present*. Additionally, we introduce *partly present* as a category to account for experiments that do suffer from a pitfall, but where the impact has been partially addressed. If a pitfall is *present* or *partly present* but acknowledged in the text, we moderate the classification as *discussed*. If the reviewers are unable to rule out the presence of a pitfall due to missing information, we mark the publication as *unclear from text*. Finally, in the special case of P10, if the pitfall *does not apply* to a paper’s setting, this is considered as a separate category.

Observations. The aggregated results from the prevalence analysis are shown in Figure 3. A bar’s color indicates the degree to which a pitfall is present, and its width shows the proportion of papers with that classification. The number of affected papers is noted at the center of the bars. The most prevalent pitfalls are sampling bias (P1) and data snooping (P3), which are at least partly present in 90 % and 73 % of the papers, respectively. In more than 50 % of the papers, we identify inappropriate threat models (P10), lab-only evaluations (P9), and inappropriate performance measures (P7) as at least partly present. *Every* paper is affected by at least three pitfalls, underlining the pervasiveness of such issues in recent computer security research. In particular, we find that dataset collection is still very challenging: some of the most realistic and expansive open datasets we have developed as a community are still imperfect (see §4.1).

Moreover, the presence of some pitfalls is more likely to be *unclear from the text* than others. We observe this for biased parameter selection (P5) when no description of the

hyperparameters or tuning procedure is given; for spurious correlations (P4) when there is no attempt to explain a model’s decisions; and for data snooping (P3) when the dataset splitting or normalization procedure is not explicitly described in the text. These issues also indicate that experimental settings are more difficult to reproduce due to a lack of information.

Feedback from authors. To foster a discussion within our community, we have contacted the authors of the selected papers and collected feedback on our findings. We conducted a survey with 135 authors for whom contact information has been available. To protect the authors’ privacy and encourage an open discussion, all responses have been anonymized.

The survey consists of a series of general and specific questions on the identified pitfalls. First, we ask the authors whether they have read our work and consider it helpful for the community. Second, for each pitfall, we collect feedback on whether they agree that (a) their publication might be affected, (b) the pitfall frequently occurs in security papers, and (c) it is easy to avoid in most cases. To quantitatively assess the responses, we use a five-point Likert scale for each question that ranges from *strongly disagree* to *strongly agree*. Additionally, we provide an option of *prefer not to answer* and allow the authors to omit questions.

We have received feedback from 49 authors, yielding a response rate of 36 %. These authors correspond to 13 of the 30 selected papers and thus represent 43 % of the considered research. Regarding the general questions, 46 (95 %) of the authors have read our paper and 48 (98 %) agree that it helps to raise awareness for the identified pitfalls. For the specific pitfall questions, the overall agreement between the authors and our findings is 63 % on average, varying depending on the security area and pitfall. All authors agree that their paper may suffer from at least one of the pitfalls. On average, they indicate that 2.77 pitfalls are present in their work with a standard deviation of 1.53 and covering all ten pitfalls.

When assessing the pitfalls in general, the authors especially agree that lab-only evaluations (92 %), the base rate fallacy (77 %), inappropriate performance measures (69 %), and sampling bias (69 %) frequently occur in security papers. Moreover, they state that inappropriate performance measures (62 %), inappropriate parameter selection (62 %), and the base rate fallacy (46 %) can be easily avoided in practice, while the other pitfalls require more effort. We provide further information on the survey in Appendix B.

In summary, we derive three central observations from this survey. First, most authors agree that there is a lack of awareness for the identified pitfalls in our community. Second, they confirm that the pitfalls are widespread in security literature and that there is a need for mitigating them. Third, a consistent understanding of the identified pitfalls is still lacking. As an example, several authors (44 %) neither agree nor disagree on whether data snooping is easy to avoid, emphasizing the importance of clear definitions and recommendations.

Takeaways. We find that all of the pitfalls introduced in §2 are pervasive in security research, affecting between 17 % and 90 % of the selected papers. Each paper suffers from at least three of the pitfalls and only 22 % of instances are accompanied by a discussion in the text. While authors may have even deliberately omitted a discussion of pitfalls in some cases, the results of our prevalence analysis overall suggest a lack of awareness in our community.

Although these findings point to a serious problem in research, we would like to remark that *all* of the papers analyzed provide excellent contributions and valuable insights. Our objective here is not to blame researchers for stepping into pitfalls but to raise awareness and increase the experimental quality of research on machine learning in security.

4 Impact Analysis

In the previous sections, we have presented pitfalls that are widespread in the computer security literature. However, so far it remains unclear how much the individual pitfalls could affect experimental results and their conclusions. In this section, we estimate the experimental impact of some of these pitfalls in popular applications of machine learning in security. At the same time, we demonstrate how the recommendations discussed in §2 help in identifying and resolving these problems. For our discussion, we consider four popular research topics in computer security:

- §4.1: mobile malware detection (P1, P4, and P7)
- §4.2: vulnerability discovery (P2, P4, and P6)
- §4.3: source code authorship attribution (P1 and P4)
- §4.4: network intrusion detection (P6 and P9)

Remark. For this analysis, we consider state-of-the-art approaches for each security domain. We remark that the results within this section do not mean to criticize these approaches specifically; we choose them as they are *representative* of how pitfalls can impact different domains. Notably, the fact that we have been able to reproduce the approaches speaks highly of their academic standard.

4.1 Mobile Malware Detection

The automatic detection of Android malware using machine learning is a particularly lively area of research. The design and evaluation of such methods are delicate and may exhibit some of the previously discussed pitfalls. In the following, we discuss the effects of sampling bias (P1), spurious correlations (P4), and inappropriate performance measures (P7) on learning-based detection in this context.

Dataset collection. A common source of recent mobile data is the *AndroZoo* project [6], which collects Android apps from a large variety of sources, including the official *GooglePlay*

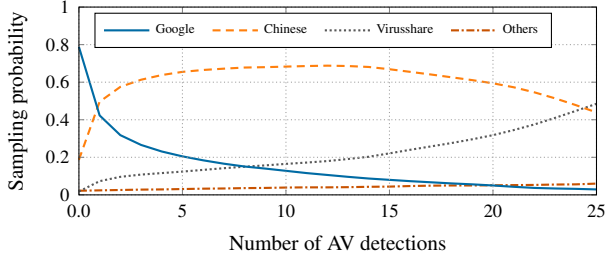


Figure 4: The probability of sampling malware from Chinese markets is significantly higher than for GooglePlay. This can lead to sampling biases in experimental setups for Android malware detection.

store and several Chinese markets. At the time of writing it includes more than 11 million Android applications from 18 different sources. As well as the samples themselves, it includes meta-information, such as the number of antivirus detections. Although AndroZoo is an excellent source for obtaining mobile apps, we demonstrate that experiments may suffer from severe sampling bias (P1) if the peculiarities of the dataset are not taken into account. Please note that the following discussion is not limited to the AndroZoo data, but is relevant for the composition of Android datasets in general.

Dataset analysis. In the first step, we analyze the data distribution of AndroZoo by considering the origin of an app and the number of antivirus detections of an Android app. For our analysis, we broadly divide the individual markets into four different origins: GooglePlay, Chinese markets, VirusShare, and all other markets.

Figure 4 shows the probability of randomly sampling from a particular origin depending on the number of antivirus detections for an app. For instance, when selecting a sample with no constraints on the number of detections, the probability of sampling from GooglePlay is roughly 80 %. If we consider a threshold of 10 detections, the probability that we randomly select an app from a Chinese market is 70 %. It is very likely that a large fraction of the benign apps in a dataset are from GooglePlay, while most of the malicious ones originate from Chinese markets, if we ignore the data distribution.

Note that this sampling bias is not limited to AndroZoo. We identify a similar sampling bias for the DREBIN dataset [9], which is commonly used to evaluate the performance of learning-based methods for Android malware detection [e.g., 9, 58, 146].

Experimental setup. To get a better understanding of this finding, we conduct experiments using two datasets: For the first dataset (D_1), we merge 10,000 benign apps from GooglePlay with 1,000 malicious apps from Chinese markets (*Anzhi* and *AppChina*). We then create a second dataset (D_2) using the same 10,000 benign applications, but combine them with 1,000 malware samples exclusively from GooglePlay. All malicious apps are detected by at least 10 virus scanners. Next, we train a linear support vector machine [47] on these datasets using two feature sets taken from state-of-the-art classifiers (DREBIN [9] and OPSEQS [91]).

Table 1: Comparison of results for two classifiers when merging benign apps from GooglePlay with Chinese malware (D_1) vs. sampling solely from GooglePlay (D_2). For both classifiers, the detection performance drops significantly when considering apps only from GooglePlay. The standard deviation of the results ranges between 0–3%.

Metric	DREBIN			OPSEQS		
	D_1	D_2	Δ	D_1	D_2	Δ
Accuracy	0.994	0.980	−1.4 %	0.972	0.948	−2.5 %
Precision	0.968	0.930	−3.9 %	0.822	0.713	−13.3 %
Recall	0.964	0.846	−12.2 %	0.883	0.734	−16.9 %
F1-Score	0.970	0.886	−8.7 %	0.851	0.722	−15.2 %
MCC [89]	0.963	0.876	−9.0 %	0.836	0.695	−16.9 %

Results. The recall (true positive rate) for DREBIN and OPSEQS drops by more than 10 % and 15 %, respectively, between the datasets D_1 and D_2 , while the accuracy is only slightly affected (see Table 1). Hence, the choice of the performance measure is crucial (P7). Interestingly, the URL *play.google.com* turns out to be one of the five most discriminative features for the benign class, indicating that the classifier has learned to distinguish the origins of Android apps, rather than the difference between malware and benign apps (P4). Although our experimental setup overestimates the classifiers’ performance by deliberately ignoring time dependencies (P3), we can still clearly observe the impact of the pitfalls. Note that the effect of temporal snooping in this setting has been demonstrated in previous work [4, 104].

4.2 Vulnerability Discovery

Vulnerabilities in source code can lead to privilege escalation and remote code execution, making them a major threat. Since the manual search for vulnerabilities is complex and time consuming, machine learning-based detection approaches have been proposed in recent years [57, 83, 141]. In what follows, we show that a dataset for vulnerability detection contains artifacts that occur only in one class (P4). We also find that VulDeePecker [83], a neural network to detect vulnerabilities, uses artifacts for classification and that a simple linear classifier achieves better results on the same dataset (P6). Finally, we discuss how the preprocessing steps proposed for VulDeePecker make it impossible to decide whether some snippets contain vulnerabilities or not (P2).

Dataset collection. For our analysis we use the dataset published by Li et al. [83], which contains source code from the National Vulnerability Database [36] and the SARD project [37]. We focus on vulnerabilities related to buffers (CWE-119) and obtain 39,757 source code snippets of which 10,444 (26 %) are labeled as containing a vulnerability.

Dataset analysis. We begin our analysis by classifying a random subset of code snippets by hand to spot possible artifacts in the dataset. We find that certain sizes of buffers seem to be present only in one class throughout the samples considered. To investigate, we extract the buffer sizes of char

Table 2: Different buffer sizes in the Vulnerability Dataset used by Li et al. [83] with their number of occurrences and relative frequency in class 0.

Buffer size	Occurrences	
	Total	In class 0
3	70	53 (75.7 %)
32	116	115 (99.1 %)
100	6,364	4,315 (67.8 %)
128	26	24 (92.3 %)
1,024	100	96 (96.0 %)

arrays that are initialized in the dataset and count the number of occurrences in each class. We report the result for class 0 (snippets without vulnerabilities) in Table 2 and observe that certain buffer sizes occur almost exclusively in this class. If the model relies on buffer sizes as discriminative features for classification, this would be a spurious correlation (P4).

Experimental setup. We train VulDeePecker [83], based on a recurrent neural network [65], to classify the code snippets automatically. To this end, we replace variable names with generic identifiers (e.g., INT2) and truncate the snippets to 50 tokens, as proposed in the paper [83]. An example of this procedure can be seen in Figure 5 where the original code snippet (top) is transformed to a generic snippet (bottom).

We use a linear Support Vector Machine (SVM) with bag-of-words features based on n -grams as a baseline for VulDeePecker. To see what VulDeePecker has learned we follow the work of Warnecke et al. [133] and use the Layerwise Relevance Propagation (LRP) method [12] to explain the predictions and assign each token a *relevance* score that indicates its importance for the classification. Figure 5 (bottom) shows an example for these scores where blue tokens favor the classification and orange ones oppose it.

Results. To see whether VulDeePecker relies on artifacts, we use the relevance values for the entire training set and extract the ten most important tokens for each code snippet. Afterwards we extract the tokens that occur most often in this top-10 selection and report the results in Table 3 in descending order of occurrence.

While the explanations are still hard to interpret for a human we notice two things: Firstly, tokens such as ‘(’, ‘]’,

```

1 data = new char[10+1];
2 char source[10+1] = SRC_STRING;
3 memmove(data, source, (strlen(source) + 1) *
  sizeof(char));

1 VAR0 = new char [ INT0 + INT1 ] ;
2 char VAR1 [ INT0 + INT1 ] = VAR2 ;
3 memmove ( VAR0 , VAR1 , ( strlen ( VAR1 ) + INT1 )
  * sizeof ( char ) ) ;

```

Figure 5: Top: Code snippet from the dataset. Bottom: Same code snippet after preprocessing steps of VulDeePecker. Coloring indicates importance towards classification according to the LRP [12] method.

Table 3: The 10 most frequent tokens across samples in the dataset.

Rank	Token	Occurrence	Rank	Token	Occurrence
1	INT1	70.8 %	6	char	38.8 %
2	(	61.1 %	7	]	32.1 %
3	*	47.2 %	8	+	31.1 %
4	INT2	45.7 %	9	VAR0	28.7 %
5	INT0	38.8 %	10	,	26.0 %

and ‘,’ are among the most important features throughout the training data although they occur frequently in code from both classes as part of function calls or array initialization. Secondly, there are many generic INT* values which frequently correspond to buffer sizes. From this we conclude that VulDeePecker is relying on combinations of artifacts in the dataset and thus suffers from spurious correlations (P4).

To further support this finding, we show in Table 4 the performance of VulDeePecker compared to an SVM and an ensemble of standard models, such as random forests and AdaBoost classifiers, trained with the *AutoSklearn* library [48]. We find that an SVM with 3-grams yields the best performance with an $18\times$ smaller model. This is interesting as overlapping but independent substrings (n -grams) are used, rather than the true sequential ordering of all tokens as for the RNN. Thus, it is likely that VulDeePecker is not exploiting relations in the sequence, but merely combines special tokens—an insight that could have been obtained by training a linear classifier (P6). Furthermore, it is noteworthy that both baselines provide significantly higher true positive rates, although the AUC-ROC of all approaches only slightly differs.

Finally, we discuss the preprocessing steps proposed by Li et al. [83] as seen in the example of Figure 5. By truncating the code snippets to a fixed length of 50, important information is lost. For example, the value of the variable SRC_STRING and thus its length is unknown to the network. Likewise, the conversion of numbers to INT0 and INT1 results in the same problem for the data variable: after the conversion it is not possible to tell how big the buffer is and whether the content fits into it or not. Depending on the surrounding code it can become impossible to say whether buffer overflows appear or not, leading to cases of label inaccuracy (P2).

4.3 Source Code Author Attribution

The task of identifying the developer based on source code is known as authorship attribution [23]. Programming habits are characterized by a variety of stylistic patterns, so that

Table 4: Performance of Support Vector Machines and VulDeePecker on unseen data. The true-positive rate is determined at 2.9 % false positives.

Model	# parameters	AUC	TPR
VulDeePecker	1.2×10^6	0.984	0.818
SVM	6.6×10^4	0.986	0.963
AutoSklearn	8.5×10^5	0.982	0.894

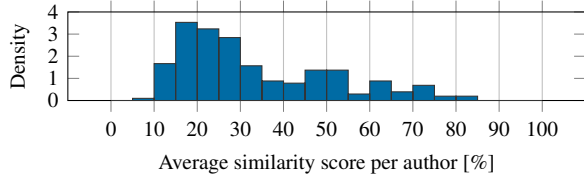


Figure 6: Shared source code over all files per author. A majority tend to copy code snippets across challenges, leading to learned artifacts.

state-of-the-art attribution methods use an expressive set of such features. These range from simple layout properties to more unusual habits in the use of syntax and control flow. In combination with sampling bias (P1), this expressiveness may give rise to spurious correlations (P4) in current attribution methods, leading to an overestimation of accuracy.

Dataset collection. Recent approaches have been tested on data from the Google Code Jam (GCJ) programming competition [2, 7, 23], where participants solve the same challenges in various rounds. An advantage of this dataset is that it ensures a classifier learns to separate stylistic patterns rather than merely overfitting to different challenges. We use the 2017 GCJ dataset [107], which consists of 1,632 C++ files from 204 authors solving the same eight challenges.

Dataset analysis. We start with an analysis of the average similarity score between all files of each respective programmer, where the score is computed by *diff*lib’s *Sequence-Matcher* [106]. Figure 6 shows that most participants copy code across the challenges, that is, they reuse personalized coding *templates*. Understandably, this results from the nature of the competition, where participants are encouraged to solve challenges quickly. These templates are often *not* used to solve the current challenges but are only present in case they might be needed. As this deviates from real-world settings, we identify a sampling bias in the dataset.

Current feature sets for authorship attribution include these templates, such that models are learned that strongly focus on them as highly discriminative patterns. However, this unused duplicate code leads to features that represent artifacts rather than coding style which are spurious correlations.

Experimental setup. Our evaluation on the impact of both pitfalls builds on the attribution methods by Abuhamad et al. [2] and Caliskan et al. [23]. Both represent the state of the art regarding performance and comprehensiveness of features.

We implement a linter tool on top of Clang, an open-source C/C++ front-end for the LLVM compiler framework, to remove unused code that is mostly present due to the templates. Based on this, we design the following three experiments: First, we train and test a classifier on the unprocessed dataset (T_b) as a baseline. Second, we remove unused code from the respective test sets (T_1), which allows us to test how much the learning methods focus on unused template code.

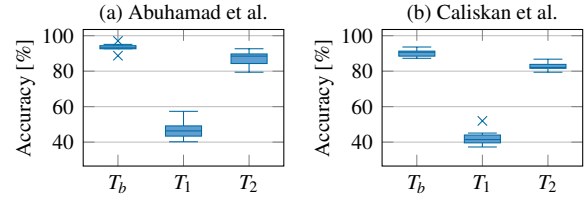


Figure 7: Accuracy of authorship attribution after considering artifacts. The accuracy drops by 48 % if unused code is removed from the test set (T_1); After retraining (T_2), the average accuracy still drops by 6 % and 7 %.

Finally, we remove unused code from the training set and re-train the classifier (T_2).

Results. Figure 7 presents the accuracy for both attribution methods on the different experiments. Artifacts have a substantial impact on the attribution accuracy. If we remove unused code from the test set (T_1), the accuracy drops by 48 % for the two approaches. This shows both systems focus considerably on the unused template code. After retraining (T_2), the average accuracy drops by 6 % and 7 % for the methods of Abuhamad et al. [2] and Caliskan et al. [23], demonstrating the reliance on artifacts for the attribution performance.

Overall, our experiments show that the impact of sampling bias and spurious correlations has been underestimated and reduces the accuracy considerably. At the same time, our results are encouraging. After accounting for artifacts, both attribution methods select features that allow for a more reliable identification. We make the sanitized dataset publicly available to foster further research in this direction.

4.4 Network Intrusion Detection

Detecting network intrusions is one of the oldest problems in security [41] and it comes at no surprise that detection of anomalous network traffic relies heavily on learning-based approaches [27, 81, 82, 93]. However, challenges in collecting real attack data [46] has often led researchers to generate synthetic data for lab-only evaluations (P9). Here, we demonstrate how this data is often insufficient for justifying the use of complex models (e.g., neural networks) and how using a simpler model as a baseline would have brought these shortcomings to light (P6).

Dataset collection. We consider the dataset released by Mirsky et al. [93], which contains a capture of Internet of Things (IoT) network traffic simulating the initial activation and propagation of the Mirai botnet malware. The packet capture covers 119 minutes of traffic on a Wi-Fi network with three PCs and nine IoT devices.

Dataset analysis. First, we analyze the transmission volume of the captured network traffic. Figure 8 shows the frequency of benign and malicious packets across the capture, divided into bins of 10 seconds. This reveals a strong signal in the packet frequency, which is highly indicative of an ongoing attack. Moreover, all benign activity seems to halt as the attack

Table 5: Comparing KITSUNE [93], an autoencoder ensemble NIDS, against a simple baseline, boxplot method [129], for detecting a Mirai infection.

Detector	AUC	TPR	TPR
		(FPR at 0.001)	(FPR at 0.000)
KITSUNE [93]	0.968	0.882	0.873
Simple Baseline [129]	0.998	0.996	0.996

commences, after 74 minutes, despite the number of devices on the network. This suggests that individual observations may have been merged and could further result in the system benefiting from spurious correlations (P4).

Experimental setup. To illustrate how severe these pitfalls are, we consider KITSUNE [93], a state-of-the-art deep learning-based intrusion detector built on an ensemble of autoencoders. For each packet, 115 features are extracted that are input to 12 autoencoders, which themselves feed to another, final autoencoder operating as the anomaly detector.

As a simple baseline to compare against KITSUNE, we choose the *boxplot method* [129], a common approach for identifying outliers. We process the packets using a 10-second sliding window and use the packet frequency per window as the sole feature. Next, we derive a lower and upper threshold from the clean calibration distribution: $\tau_{low} = Q_1 - 1.5 \cdot IQR$ and $\tau_{high} = Q_3 + 1.5 \cdot IQR$. During testing, packets are marked as benign if the sliding window’s packet frequency is between τ_{low} and τ_{high} , and malicious otherwise. In Figure 8, these thresholds are shown by the dashed gray lines.

Results. The classification performance of the autoencoder ensemble compared to the boxplot method is shown in Table 5. While the two approaches perform similarly in terms of ROC AUC, the simple boxplot method outperforms the autoencoder ensemble at low false-positive rates (FPR). As well as its superior performance, the boxplot method is exceedingly lightweight compared to the feature extraction and test procedures of the ensemble. This is especially relevant as the ensemble is designed to operate on resource-constrained devices with low latency (e.g., IoT devices).

Note this experiment does not intend to show that the boxplot method can detect an instance of Mirai operating in the wild, nor that KITSUNE is incapable of detecting other attacks, but to demonstrate that an experiment without an appropriate baseline (P6) is *insufficient to justify the complexity and overhead of the ensemble*. The success of the boxplot method also shows how simple methods can reveal issues with data generated for lab-only evaluations (P9). In the Mirai dataset the infection is overly conspicuous; an attack in the wild would likely be represented by a tiny proportion of network traffic.

4.5 Takeaways

The four case studies clearly demonstrate the impact of the considered pitfalls across four distinct security scenarios. Our findings show that subtle errors in the design and experimen-

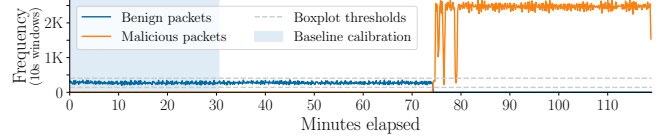


Figure 8: Frequency of benign vs malicious packets in the Mirai dataset [93]. The Gray dashed lines show the thresholds that define normal traffic calculated using the simple baseline (*boxplot method* [129]). The span of clean data used for calibration is highlighted by the light blue shaded area.

tal setup of an approach can result in misleading or erroneous results. Despite the overall valuable contributions of the research, the frequency and severity of pitfalls identified in top papers clearly indicate that significantly more awareness is needed. Additionally, we show how pitfalls apply across multiple domains, indicating a general problem that cannot be attributed to only one of the security areas.

5 Limitations and Threats to Validity

The preceding identification and analysis of common pitfalls in the security literature has been carried out with utmost care. However, there are some limitations that are naturally inherent to this kind of work. Even though these do not affect the overall conclusion of our analysis, we discuss them in the following for the sake of completeness.

Pitfalls. Although some pitfalls may seem obvious at first, our prevalence analysis indicates the opposite. This lack of awareness obstructs progress, and it will persist until addressed by the community. Furthermore, we cannot cover all ten pitfalls in detail, as our focus is on a comprehensive overview. Finally, some pitfalls cannot always be prevented, such as sampling bias, label inaccuracy, or lab-only settings. For example, it is likely not possible to test an attack in a real environment due to ethical considerations. In such cases, simulation is the only option. As outlined in §2, corrective measures may even be an open problem, yet awareness of pitfalls is a first step towards amending experimental practices and ultimately devising novel methods for mitigating them.

Prevalence analysis. For the prevalence analysis, we skimmed all papers of top security conferences in the last 10 years and identified 30 papers that use machine learning prominently (e.g., mentioned in the abstract or introduction). Even though this selection process is not entirely free from bias, the identified pitfalls are typical for this research branch and the respective papers are often highly cited.

Moreover, a pitfall is only counted if its presence is clear from the text or the associated artifacts, such as code or data. Otherwise, we decide in favor of the paper and consider a pitfall as not present. Despite this conservative assignment, our analysis underlines the prevalence of pitfalls.

Impact analysis. Four exemplary research works are chosen from security areas in which the authors of this paper

have also published research. This biased selection, however, should be acceptable, as we intend to empirically demonstrate how pitfalls can affect experimental results.

6 Related Work

Our study is the first to *systematically* and *comprehensively* explore pitfalls when applying machine learning to security. It complements a series of research on improving experimental evaluations in general. In the following, we briefly review this related work and point out key differences.

Security studies. Over the last two decades, there have been several studies on improving experiments in specific security domains. For example, Axelsson [11], McHugh [90], and Cardenas et al. [24] investigate issues with the evaluation of intrusion detection systems, covering special cases of sampling bias (P1), the base rate fallacy (P8), and inappropriate performance measures (P7). Sommer and Paxson [119] extend this work and specifically focus on the application of machine learning for network intrusion detection. They identify further issues, such as semantic gaps with anomaly detection (P4) and unrealistic evaluation baselines (P6).

In a similar strain of research, Rossow et al. [112] derive guidelines for conducting experiments with malware. Although this study does not investigate machine learning explicitly, it points to experimental problems related to some of the issues discussed in this paper. The study is expanded upon by a series of work examining variants of sampling bias in malware analysis (P1), such as temporally inconsistent data splits and labels [e.g., 4, 92, 104, 144] as well as unrealistic goodware-to-malware ratios [e.g., 5, 104]. Aghakhani et al. [3] study the limits of static features for malware classification in the presence of packed samples.

Das et al. [35] show that security defenses relying on hardware performance counters are ineffective in realistic settings (P9). Similarly, for privacy-preserving machine learning, Oya et al. [98] find that most location privacy approaches fail when applied to real-world distributions (P9). For authentication, Sugrim et al. [123] propose appropriate measures to evaluate learning-based authentication systems (P7), and finally, for system security, van der Kouwe et al. [130] point to frequent benchmarking flaws (P1, P6, and P7).

Our study builds on this research but provides an orthogonal and comprehensive view of the problem. Instead of focusing on specific domains, we are the first to *generally* explore pitfalls and recommendations when applying machine learning in computer security. Hence, our work is not limited to certain problems but applicable to all security domains.

Adversarial learning studies. Another branch of research has focused on attacking and defending learning algorithms [18, 39, 101]. While a number of powerful attacks have emerged from this research such as evasion, poisoning, and inference attacks, the corresponding defenses have of-

ten suffered from limited robustness [10]. To counteract this imbalance, Carlini et al. [26] identify several pitfalls that affect the evaluation of defenses and discuss recommendations on how to avoid them. In a similar vein, Biggio et al. [21] propose a framework for security evaluations of pattern classifiers under attack. Both works are closely related to pitfall P10 and provide valuable hints for evaluating the robustness of defenses. However, while we also argue that smart and adaptive adversaries must always be considered when proposing learning-based solutions in security, our study is more general.

Machine learning studies. Finally, a notable body of work has explored recommendations for the general use of machine learning. This research includes studies on different forms of sampling bias and dataset shift [94, 124, 128] as well as on the general implications of biased parameter selection [63], data snooping [76], and inappropriate evaluation methods [38, 52, 61]. An intuitive overview of issues in applied statistics is provided by Reinhart [109].

Our work builds on this analysis; however, we focus exclusively on the impact of pitfalls prevalent in security. Consequently, our study and its recommendations are tailored to the needs of the security community, and aim to push forward the state of the art in learning-based security systems.

7 Conclusion

We identify and systematically assess ten subtle pitfalls in the use of machine learning in security. These issues can affect the validity of research and lead to overestimating the performance of security systems. We find that these pitfalls are prevalent in security research, and demonstrate the impact of these pitfalls in different security applications. To support researchers in avoiding them, we provide recommendations that are applicable to all security domains, from intrusion and malware detection to vulnerability discovery.

Ultimately, we strive to improve the scientific quality of empirical work on machine learning in security. A decade after the seminal study of Sommer and Paxson [119], we again encourage the community to reach *outside the closed world* and explore the challenges and chances of embedding machine learning in real-world security systems.

Additional material

For interested readers, we provide supplementary material for the paper at <http://dodo-mlsec.org>.

Acknowledgements

The authors wish to thank the anonymous reviewers for their insightful and constructive comments on this paper. Also, we thank Melanie Volkamer and Sascha Fahl for their valuable feedback on the study design. Furthermore, we like

to thank Christopher J. Anders for his helpful suggestions on a previous version of the paper. The authors gratefully acknowledge funding from the German Federal Ministry of Education and Research (BMBF) as BIFOLD – Berlin Institute for the Foundations of Learning and Data (ref. 01IS18025A and ref 01IS18037A), by the Helmholtz Association (HGF) within topic “46.23 Engineering Secure Systems”, and by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy EXC 2092 CASA-390781972, and the projects 456292433; 456292463; 393063728. Moreover, we acknowledge that this research has been partially sponsored by the UK EP/P009301/1 EPSRC research grant.

References

- [1] Y. S. Abu-Mostafa, M. Magdon-Ismael, and H.-T. Lin. *Learning From Data*, chapter 5. AMLBook, 2012.
- [2] M. Abuhamad, T. AbuHmed, A. Mohaisen, and D. Nyang. Large-scale and language-oblivious code authorship identification. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2018.
- [3] H. Aghakhani, F. Gritti, F. Mecca, M. Lindorfer, S. Ortolani, D. Balzarotti, G. Vigna, and C. Kruegel. When Malware is Packin’ Heat: Limits of Machine Learning Classifiers Based on Static Analysis Features. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2020.
- [4] K. Allix, T. F. Bissyandé, J. Klein, and Y. L. Traon. Are your training datasets yet relevant? - an investigation into the importance of timeline in machine learning-based malware detection. In *Engineering Secure Software and Systems (ES-SoS)*, 2015.
- [5] K. Allix, T. F. Bissyandé, Q. Jérôme, J. Klein, Y. Le Traon, et al. Empirical assessment of machine learning-based malware detectors for android. *Empirical Software Engineering*, 2016.
- [6] K. Allix, T. F. Bissyandé, J. Klein, and Y. Le Traon. Androzoo: Collecting millions of android apps for the research community. In *Proc. of the Int. Conference on Mining Software Repositories (MSR)*, 2016.
- [7] B. Alsulami, E. Dauber, R. E. Harang, S. Mancoridis, and R. Greenstadt. Source code authorship attribution using long short-term memory based networks. In *Proc. of European Symposium on Research in Computer Security (ESORICS)*, 2017.
- [8] D. Andriesse, J. Slowinska, and H. Bos. Compiler-agnostic function detection in binaries. In *Proc. of IEEE European Symposium on Security and Privacy (EuroS&P)*, 2017.
- [9] D. Arp, M. Spreitzerbarth, M. Hübner, H. Gascon, and K. Rieck. Drebin: Efficient and explainable detection of Android malware in your pocket. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2014.
- [10] A. Athalye, N. Carlini, and D. Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. *Proc. of Int. Conference on Machine Learning (ICML)*, 2018.
- [11] S. Axelsson. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security (TISSEC)*, Aug. 2000.
- [12] S. Bach, A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, and W. Samek. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLoS ONE*, July 2015.
- [13] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *Proc. of International Conference on Learning Representations (ICLR)*, 2015.
- [14] T. Bao, J. Burket, M. Woo, R. Turner, and D. Brumley. BYTEWEIGHT: Learning to recognize functions in binary code. In *Proc. of USENIX Security Symposium*, 2014.
- [15] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro. Transcending transcend: Revisiting malware classification with conformal evaluation. *arXiv:2010.03856v1*, 2020.
- [16] E. Bareinboim, J. Tian, and J. Pearl. Recovering from selection bias in causal and statistical inference. In *Proc. of the AAAI Conference on Artificial Intelligence (AAAI)*, 2014.
- [17] D. Barradas, N. Santos, and L. E. T. Rodrigues. Effective detection of multimedia protocol tunneling using machine learning. In *Proc. of USENIX Security Symposium*, 2018.
- [18] B. Biggio and F. Roli. Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition*, 2018.
- [19] B. Biggio, B. Nelson, and P. Laskov. Support vector machines under adversarial label noise. In *Proc. of Asian Conference on Machine Learning (ACML)*, 2011.
- [20] B. Biggio, I. Corona, D. Maiorca, B. Nelson, N. Šrđić, P. Laskov, G. Giacinto, and F. Roli. Evasion attacks against machine learning at test time. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. Springer, 2013.
- [21] B. Biggio, G. Fumera, and F. Roli. Security evaluation of pattern classifiers under attack. *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, 2014.
- [22] Y. Boshmaf, D. Logothetis, G. Siganos, J. Lería, J. Lorenzo, M. Ripeanu, and K. Beznosov. Integro: Leveraging victim prediction for robust fake account detection in osns. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2015.
- [23] A. Caliskan, R. Harang, A. Liu, A. Narayanan, C. R. Voss, F. Yamaguchi, and R. Greenstadt. De-anonymizing programmers via code stylometry. In *Proc. of USENIX Security Symposium*, 2015.
- [24] A. A. Cardenas, J. S. Baras, and K. Seamon. A framework for the evaluation of intrusion detection systems. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2006.
- [25] N. Carlini and D. A. Wagner. Towards evaluating the robustness of neural networks. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2017.
- [26] N. Carlini, A. Athalye, N. Papernot, W. Brendel, J. Rauber, D. Tsipras, I. J. Goodfellow, A. Madry, and A. Kurakin. On evaluating adversarial robustness. *CoRR*, abs/1902.06705, 2019.
- [27] V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Computing Surveys (CSUR)*, 2009.
- [28] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research (JAIR)*, 16, 2002.
- [29] D. Chicco and G. Jurman. The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21, 2020.
- [30] C. Chio and D. Freeman. *Machine Learning and Security: Protecting Systems with Data and Algorithms*. O’Reilly Media, Inc., 2018.
- [31] K. Cho, B. van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proc. of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.
- [32] Z. L. Chua, S. Shen, P. Saxena, and Z. Liang. Neural nets can learn function type signatures from binaries. In *Proc. of USENIX Security Symposium*, 2017.
- [33] C. Cortes, M. Mohri, M. Riley, and A. Rostamizadeh. Sample selection bias correction theory. In *Proc. of the Int. Conference on Algorithmic Learning Theory (ALT)*, 2008.
- [34] C. Curtissinger, B. Livshits, B. Zorn, and C. Seifert. Zozzle: Fast and precise in-browser javascript malware detection. In *Proc. of USENIX Security Symposium*, 2011.
- [35] S. Das, J. Werner, M. Antonakakis, M. Polychronakis, and F. Monrose. Sok: The challenges, pitfalls, and perils of using hardware performance counters for security. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2019.
- [36] N. V. Database. <https://nvd.nist.gov/>. (last visited Oct. 15, 2020).
- [37] S. A. R. Dataset. <https://samate.nist.gov/SRD/index.php>. (last visited Oct. 15, 2020).
- [38] J. Davis and M. Goadrich. The relationship between precision-recall and roc curves. In *Proc. of Int. Conference on Machine Learning (ICML)*, 2006.
- [39] E. De Cristofaro. An overview of privacy in machine learning. *arXiv:2005.08679*, 2020.
- [40] A. Demontis, M. Melis, B. Biggio, D. Maiorca, D. Arp, K. Rieck, I. Corona, G. Giacinto, and F. Roli. Yes, machine learning can be more secure! a case study on android malware detection. *IEEE Transactions on Dependable and Secure Computing (TDSC)*, 2019.
- [41] D. E. Denning. An intrusion-detection model. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 1986.
- [42] S. H. H. Ding, B. C. M. Fung, and P. Charland. Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and

- compiler optimization. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2019.
- [43] M. Du, F. Li, G. Zheng, and V. Srikumar. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2017.
- [44] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton. Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2012.
- [45] W. Enck, M. Ongtang, and P. D. McDaniel. On lightweight mobile phone application certification. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2009.
- [46] G. Engelen, V. Rimmer, and W. Joosen. Troubleshooting an intrusion detection dataset: the CICSIDS2017 case study. In *2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2021.
- [47] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin. LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research (JMLR)*, 9, 2008.
- [48] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems (NIPS)*, 2015.
- [49] F. Fischer, K. Böttinger, H. Xiao, C. Stransky, Y. Acar, M. Backes, and S. Fahl. Stack overflow considered harmful? the impact of copy&paste on android application security. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2017.
- [50] F. Fischer, H. Xiao, C.-Y. Kao, Y. Stachelscheid, B. Johnson, D. Razar, P. Fawkesley, N. Buckley, K. Böttinger, P. Muntean, and J. Grossklags. Stack overflow considered helpful! deep learning security nudges towards stronger cryptography. In *Proc. of USENIX Security Symposium*, 2019.
- [51] G. Forman. A pitfall and solution in multi-class feature selection for text classification. In *Proc. of Int. Conference on Machine Learning (ICML)*, 2004.
- [52] G. Forman and M. Scholz. Apples-to-apples in cross-validation studies: Pitfalls in classifier performance measurement. *SIGKDD Explor. Newsl.*, 2010.
- [53] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff. A sense of self for unix processes. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 1996.
- [54] M. Fredrikson, S. Jha, M. Christodorescu, R. Sailer, and X. Yan. Synthesizing near-optimal malware specifications from suspicious behaviors. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2010.
- [55] B. Frenay and M. Verleysen. Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 2014.
- [56] P. Ghosh. AAS: Machine learning 'causing science crisis'. <https://www.bbc.co.uk/news/science-environment-47267081>, 2019. (last visited Oct. 15, 2020).
- [57] G. Grieco, G. L. Grinblat, L. Uzal, S. Rawat, J. Feist, and L. Mounier. Toward large-scale vulnerability discovery using machine learning. In *Proc. of ACM Conference on Data and Applications Security and Privacy (CODASPY)*, 2016.
- [58] K. Grosse, N. Papernot, P. Manoharan, M. Backes, and P. D. McDaniel. Adversarial examples for malware detection. In *Proc. of European Symposium on Research in Computer Security (ESORICS)*, 2017.
- [59] W. Guo, D. Mu, J. Xu, P. Su, G. Wang, and X. Xing. Lemna: Explaining deep learning based security applications. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2018.
- [60] H. Han, W.-Y. Wang, and B.-H. Mao. Borderline-smote: A new over-sampling method in imbalanced data sets learning. In *Advances in Intelligent Computing*, 2005.
- [61] D. J. Hand. Measuring Classifier Performance: a Coherent Alternative to the Area Under the ROC Curve. *Machine Learning*, 2009.
- [62] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proc. of IEEE Conference on Computer Vision and Pattern (CVPR)*, 2016.
- [63] M. L. Head, L. Holman, R. Lanfear, A. T. Kahn, and M. D. Jennions. The extent and consequences of p-hacking in science. *PLOS Biology*, 2015.
- [64] J. J. Heckman. Sample Selection Bias as a Specification Error. *Econometrica*, 47(1):153–161, 1979.
- [65] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 1997.
- [66] S. Hooker, D. Erhan, P.-J. Kindermans, and B. Kim. A benchmark for interpretability methods in deep neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2019.
- [67] M. Hurier, G. Suarez-Tangil, S. K. Dash, T. F. Bissyandé, Y. L. Traon, J. Klein, and L. Cavallaro. Euphony: harmonious unification of cacophonous anti-virus vendor labels for android malware. In *Proc. of the ACM International Conference on Mining Software Repositories (MSR)*, 2017.
- [68] U. Iqbal, P. Snyder, S. Zhu, B. Livshits, Z. Qian, and Z. Shafiq. Adgraph: A graph-based approach to ad and tracker blocking. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2020.
- [69] J. Jang, D. Brumley, and S. Venkataraman. Bitshred: feature hashing malware for scalable triage and semantic analysis. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2011.
- [70] H. Jin, Q. Song, and X. Hu. Auto-keras: An efficient neural architecture search system. In *Proc. of the ACM SIGKDD International Conference On Knowledge Discovery and Data Mining (KDD)*, 2019.
- [71] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouredinov, and L. Cavallaro. Transcend: Detecting concept drift in malware classification models. In *Proc. of USENIX Security Symposium*, 2017.
- [72] M. Juarez, S. Afroz, G. Acar, C. Diaz, and R. Greenstadt. A critical evaluation of website fingerprinting attacks. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2014.
- [73] A. Kerckhoffs. *La cryptographie militaire*. 1883.
- [74] A. Kharraz, W. K. Robertson, and E. Kirda. Surveyance: Automatically detecting online survey scams. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2018.
- [75] P.-J. Kindermans, S. Hooker, J. Adebayo, M. Alber, K. T. Schütt, S. Dähne, D. Erhan, and B. Kim. The (un)reliability of saliency methods. In *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. 2019.
- [76] J. Komiya and T. Maehara. A simple way to deal with cherry-picking. *arXiv:1810.04996*, 2018.
- [77] K. Krippendorff. *Content analysis: An introduction to its methodology*. Sage publications, 2018.
- [78] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet: Classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2012.
- [79] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, and K.-R. Müller. Unmasking Clever Hans predictors and assessing what machines really learn. *Nature Communications*, 10(1), 2019.
- [80] E. Lee, J. Woo, H. Kim, A. Mohaisen, and H. K. Kim. You are a game bot!: Uncovering game bots in MMORPGs via self-similarity in the wild. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2016.
- [81] W. Lee and S. J. Stolfo. Data mining approaches for intrusion detection. In *Proc. of USENIX Security Symposium*, 1998.
- [82] Y. Li, J. Xia, S. Zhang, J. Yan, X. Ai, and K. Dai. An efficient intrusion detection system based on support vector machines and gradually feature removal method. *Expert Systems with Applications*, 2012.
- [83] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong. Vuldeepecker: A deep learning-based system for vulnerability detection. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2018.
- [84] J. Liang, W. Guo, T. Luo, V. Honavar, G. Wang, and X. Xing. FARE: Enabling Fine-grained Attack Categorization under Low-quality Labeled Data. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2021.
- [85] Z. C. Lipton, Y. Wang, and A. J. Smola. Detecting and correcting for label shift with black box predictors. In *Proc. of Int. Conference on Machine Learning (ICML)*, 2018.
- [86] A. Liu and B. D. Ziebart. Robust classification under sample selection bias. In *Advances in Neural Information Processing Systems (NIPS)*, 2014.
- [87] F. Maggi, W. Robertson, C. Kruegel, and G. Vigna. Protecting a moving target: Addressing web application concept drift. In *Proc. of International Symposium on Recent Advances in Intrusion Detection (RAID)*, 2009.
- [88] E. Mariconti, L. Onwuzurike, P. Andriotis, E. D. Cristofaro, G. J. Ross, and G. Stringhini. Mamadroid: Detecting android malware by building markov chains of behavioral models. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2017.
- [89] B. Matthews. Comparison of the predicted and observed secondary structure of t4 phage lysozyme. *Biochimica et Biophysica Acta (BBA) - Protein Structure*, 405(2), 1975.
- [90] J. McHugh. Testing intrusion detection systems: A critique of the 1998 and 1999 darpa intrusion detection system evaluations as performed by lincoln laboratory. *ACM Transactions on Information and System Security (TISSEC)*, 2000.
- [91] N. McLaughlin, J. Martinez del Rincon, B. Kang, S. Yerima, P. Miller, S. Sezer,

- Y. Safaei, E. Trickett, Z. Zhao, A. Doupé, and G. Joon Ahn. Deep android malware detection. In *Proc. of ACM Conference on Data and Applications Security and Privacy (CODASPY)*, 2017.
- [92] B. Miller, A. Kantchelian, M. C. Tschantz, S. Afroz, R. Bachwani, R. Faizulabhooy, L. Huang, V. Shankar, T. Wu, G. Yiu, A. D. Joseph, and J. D. Tygar. Reviewer integration and performance measurement for malware detection. In *Proc. of Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA)*, 2016.
- [93] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2018.
- [94] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera. A unifying view on dataset shift in classification. *Pattern Recognition*, 2012.
- [95] S. Nilizadeh, F. Labreche, A. Sedighian, A. Zand, J. M. Fernandez, C. Kruegel, G. Stringhini, and G. Vigna. POISED: spotting twitter spam off the beaten paths. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2017.
- [96] C. G. Northcutt, L. Jiang, and I. L. Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research (JAIR)*, 70, 2021.
- [97] Z. Ovaisi, R. Ahsan, Y. Zhang, K. Vasilaky, and E. Zheleva. Correcting for selection bias in learning-to-rank systems. In *Proc. of the International World Wide Web Conference (WWW)*, page 1863–1873, 2020.
- [98] S. Oya, C. Troncoso, and F. Pérez-González. Rethinking location privacy for unknown mobility behaviors. In *Proc. of IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.
- [99] M. Palatucci, D. Pomerleau, G. Hinton, and T. M. Mitchell. Zero-shot learning with semantic output codes. In *Advances in Neural Information Processing Systems (NIPS)*, 2009.
- [100] A. Panchenko, F. Lanze, A. Zinnen, M. Henze, J. Pennekamp, K. Wehrle, and T. Engel. Website fingerprinting at Internet scale. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2016.
- [101] N. Papernot, P. McDaniel, A. Sinha, and M. P. Wellman. SoK: Security and privacy in machine learning. In *Proc. of IEEE European Symposium on Security and Privacy (EuroS&P)*, Apr. 2018.
- [102] V. Paxson. Bro: A system for detecting network intruders in real-time. In *Proc. of USENIX Security Symposium*, 1998.
- [103] J. Pearl, G. Madelyn, and N. P. Jewell. *Causal inference in statistics : a primer*. Wiley, 2016.
- [104] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro. TESSER-ACT: Eliminating Experimental Bias in Malware Classification across Space and Time. In *Proc. of USENIX Security Symposium*, 2019.
- [105] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro. Intriguing properties of adversarial ml attacks in the problem space. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2020.
- [106] Python Software Foundation. `difflib` – helpers for computing deltas. <https://docs.python.org/3/library/difflib.html>. (last visited Sep. 1, 2021).
- [107] E. Quiring, A. Maier, and K. Rieck. Misleading authorship attribution of source code using adversarial learning. In *Proc. of USENIX Security Symposium*, 2019.
- [108] E. Quiring, D. Klein, D. Arp, M. Johns, and K. Rieck. Adversarial preprocessing: Understanding and preventing image-scaling attacks in machine learning. In *Proc. of USENIX Security Symposium*, 2020.
- [109] A. Reinhart. *Statistics Done Wrong: The Woefully Complete Guide*. No Starch Press, 2015.
- [110] V. Rimmer, D. Preuveneers, M. Juárez, T. van Goethem, and W. Joosen. Automated website fingerprinting through deep learning. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2018.
- [111] M. Roesch. Snort - lightweight intrusion detection for networks. In *Proc. of the USENIX Conference on System Administration (LISA)*, 1999.
- [112] C. Rossow, C. Dietrich, C. Gier, C. Kreibich, V. Paxson, N. Pohlmann, H. Bos, and M. van Steen. Prudent practices for designing malware experiments: Status quo and outlook. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2012.
- [113] Y. Shen, E. Mariconti, P. Vervier, and G. Stringhini. Tiresias: Predicting security events through deep learning. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2018.
- [114] E. C. R. Shin, D. Song, and R. Moazzezi. Recognizing functions in binaries with neural networks. In *Proc. of USENIX Security Symposium*, 2015.
- [115] X. Shu, D. Yao, and N. Ramakrishnan. Unearthing stealthy program attacks buried in extremely long execution paths. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2015.
- [116] A. Shusterman, L. Kang, Y. Haskal, Y. Meltser, P. Mittal, Y. Oren, and Y. Yarom. Robust website fingerprinting through the cache occupancy channel. In *Proc. of USENIX Security Symposium*, 2019.
- [117] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *Proc. of International Conference on Learning Representations (ICLR)*, 2015.
- [118] M. Sokolova and G. Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing and Management*, 45(4), 2009.
- [119] R. Sommer and V. Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2010.
- [120] J. Song, S. Lee, and J. Kim. Crowdtarg: Target-based detection of crowdturfing in online social networks. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2015.
- [121] N. Srđić and P. Laskov. Detection of malicious PDF files based on hierarchical document structure. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2013.
- [122] P. Stock and M. Cissé. Convnets and imagenet beyond accuracy: Understanding mistakes and uncovering biases. In *Proc. of the European Conference on Computer Vision (ECCV)*, 2018.
- [123] S. Sugrim, C. Liu, M. McLean, and J. Lindqvist. Robust performance metrics for authentication systems. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2019.
- [124] H. Suresh and J. V. Guttat. A framework for understanding sources of harm throughout the machine learning life cycle. *arXiv:1901.10002*, 2021.
- [125] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2014.
- [126] K. M. C. Tan and R. A. Maxion. "why 6?" defining the operational limits of stide, an anomaly-based intrusion detector. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2002.
- [127] R. Tomsett, D. Harborne, S. Chakraborty, P. Gurram, and A. Preece. Sanity checks for saliency metrics. In *Proc. of the AAAI Conference on Artificial Intelligence (AAAI)*, 2020.
- [128] A. Torralba and A. A. Efros. Unbiased look at dataset bias. In *Proc. of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011.
- [129] J. W. Tukey. *Exploratory data analysis*. Addison-Wesley series in behavioral science : quantitative methods. Addison-Wesley, 1977.
- [130] E. van der Kouwe, G. Heiser, D. Andriess, H. Bos, and C. Giuffrida. SoK: Benchmarking Flaws in Systems Security. In *Proc. of IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.
- [131] N. Šrđić and P. Laskov. Practical evasion of a learning-based classifier: A case study. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2014.
- [132] K. Wang, J. J. Parekh, and S. J. Stolfo. Anagram: A content anomaly detector resistant to mimicry attack. In *Proc. of Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*, 2006.
- [133] A. Warnecke, D. Arp, C. Wressnegger, and K. Rieck. Evaluating explanation methods for deep learning in security. In *Proc. of IEEE European Symposium on Security and Privacy (EuroS&P)*, 2020.
- [134] F. Wei, Y. Li, S. Roy, X. Ou, and W. Zhou. Deep ground truth analysis of current android malware. *Proc. of Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA)*, 2017.
- [135] K. R. Weiss, T. M. Khoshgoftaar, and D. Wang. A survey of transfer learning. *Journal of Big Data*, 3:9, 2016.
- [136] D. H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 1996.
- [137] S. C. Wong, A. Gatt, V. Stamatescu, and M. D. McDonnell. Understanding data augmentation for classification: When to warp? In *Int. Conference on Digital Image Computing: Techniques and Applications (DICTA)*, 2016.
- [138] S. Xi, S. Yang, X. Xiao, Y. Yao, Y. Xiong, F. Xu, H. Wang, P. Gao, Z. Liu, F. Xu, and J. Lu. Deepintent: Deep icon-behavior learning for detecting intention-behavior discrepancy in mobile apps. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2019.
- [139] J. Xu, Y. Li, and R. H. Deng. Differential training: A generic framework to reduce label noises for android malware detection. In *Proc. of Network and Distributed System Security Symposium (NDSS)*, 2021.

- [140] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song. Neural network-based graph embedding for cross-platform binary code similarity detection. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2017.
- [141] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck. Modeling and discovering vulnerabilities with code property graphs. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2014.
- [142] F. Yamaguchi, A. Maier, H. Gascon, and K. Rieck. Automatic inference of search patterns for taint-style vulnerabilities. In *Proc. of IEEE Symposium on Security and Privacy (S&P)*, 2015.
- [143] B. Zadrozny. Learning and evaluating classifiers under sample selection bias. In *Proc. of Int. Conference on Machine Learning (ICML)*, 2004.
- [144] S. Zhu, J. Shi, L. Yang, B. Qin, Z. Zhang, L. Song, and G. Wang. Measuring and modeling the label dynamics of online anti-malware engines. In *Proc. of USENIX Security Symposium*, 2020.
- [145] Y. Zhu, D. Xi, B. Song, F. Zhuang, S. Chen, X. Gu, and Q. He. Modeling users’ behavior sequences with hierarchical explainable network for cross-domain fraud detection. In *Proc. of the International World Wide Web Conference (WWW)*, 2020.
- [146] Z. Zhu and T. Dumitraş. Featuresmith: Automatically engineering features for malware detection by mining the security literature. In *Proc. of ACM Conference on Computer and Communications Security (CCS)*, 2016.
- [147] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He. A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1), 2021.

A Appendix: Pitfalls

As a supplement to this paper, we provide further details on the identified pitfalls and our recommendations in this section.

Label inaccuracy. Noisy labels are a common problem in machine learning and a source of bias. In contrast to sampling bias, however, there exist different, practical methods for mitigating label noise [e.g., 96, 139]. To demonstrate this mitigation, we employ the readily available method by Northcutt et al. [96] that cleans noisy instances in the training data. We follow the setup from Xu et al. [139] and randomly flip 9.7 % of the labels in the DREBIN training dataset. We then train an SVM on three datasets: the correctly-labelled dataset, its variant with noisy labels, and the cleansed dataset.

Table 6 shows the F1-score, Precision, and Recall for the three datasets. Due to label noise, the F1-score drops from 0.95 to 0.73 on the second dataset. Yet, it increases to 0.93 once data cleansing is applied. This result is comparable to the method by Xu et al. [139] who report an F1-score of 0.84 after repairing labels. In addition, we check the detection performance of noisy labels. 84 % of the flipped labels are correctly detected, while only 0.2 % of the original labels are falsely flagged as incorrect. Our experiment indicates that available methods for reducing label noise can provide sufficient quality to mitigate label inaccuracy in practice.

Table 6: Performance with label noise in the DREBIN dataset

Scenario	F1-Score	Precision	Recall
Correctly-labelled dataset	0.955	0.900	0.928
Noisy dataset	0.727	0.340	0.942
Cleansed dataset	0.933	0.889	0.855

Data snooping. As discussed in §2, there exist several variants of data snooping where information that is not available in practice is unintentionally used in the learning process. Table 8 provides a list of common types for test, temporal and spatial snooping to better illustrate these cases. We recommend using this table as a starting point when vetting a machine-learning workflow for the presence of data snooping.

Spurious correlations. Various extraneous factors, including sampling bias and confounding bias [16, 103], can introduce spurious correlations. In the case of confounding bias, a so-called confounder is present that coincidentally correlates with the task to solve. Depending on the used features, the confounder introduces artifacts that lead to false associations. In the case of sampling bias, the correlations result from differences between the sampled data and the true underlying data distribution.

Spurious correlations are challenging to identify, as they depend on the application domain and the concrete objective of the learning-based system. In one setting a correlation might be a valid signal, whereas in another it spuriously creates an artificial shortcut leading to over-estimated results. Consequently, we recommend systematically analyzing possible factors that can introduce these correlations. In some cases, it is then possible to explicitly control for unwanted extraneous factors that introduce spurious correlations, thus eliminating their impact on the experimental outcome. In other disciplines, different techniques have been proposed to achieve this goal [e.g., 16, 64, 86, 97, 143], which, however, often build on information not available to security practitioners. For instance, several methods [e.g., 64, 143] can correct sampling bias if the selection probability for each observation is known or can be estimated. In security research this is rarely the case.

As a remedy, we encourage the community to continuously check for extraneous factors that affect the performance of learning-based systems in experiments. However, this is a non-trivial task, as the factors contributing to the correlations are highly domain-specific. As recommended in §2, explanation techniques for machine learning can be a powerful tool in this setting to enable tracing predictions back to individual features, thereby exposing the learned correlations.

Sampling bias. Often it is extremely difficult to acquire representative data and thus some bias is unavoidable. As an example of how to tackle this problem, we investigate this pitfall for Android malware detection. In particular, we control

Table 7: Detection performance on data from different origins

Origin	F1-Score	Precision	Recall
GooglePlay	0.879	0.914	0.846
Anzhi	0.838	0.881	0.801
AppChina	0.807	0.858	0.762
AndroZoo	0.885	0.922	0.852

Table 8: Overview of data snooping groups and types

Group	Types	Description
Test Snooping	Preparatory work	If the test set is used for any experiments except for the evaluation of the final model, the learning setup benefits from additional knowledge that would not be available in practice. This includes steps to find features or limit the number of features through feature selection on the entire dataset.
	K-fold cross-validation	Another type of snooping occurs if researchers tune the hyperparameters by using k-fold cross-validation with the final test set for evaluation, and report these results.
	Normalization	Normalization factors, such as tf-idf, are computed on the complete dataset, i.e., before splitting the dataset into training and test set.
	Embeddings	Similarly, embeddings for deep neural networks are derived from the complete dataset, instead of just using the training data.
Temporal Snooping	Time dependency	Time dependencies within the data are not considered, so that samples are detected with features that would not be available at training time in a realistic setting (e.g., features of new malware variants [104]).
	Aging datasets	The usage of well-known datasets from prior work can also introduce a bias. Researchers may implicitly incorporate prior knowledge by using previous insights from these publicly available datasets.
Selective Snooping	Cherry-picking	Data is cleaned based on information that is usually not available in practice. For instance, applications are filtered out that are not detected by a sufficiently large number of AV scanners.
	Survivorship bias	A group of samples is already filtered out. This bias overlaps with sampling bias (P1). For example, using only applications, which a dynamic analysis system can successfully process and removing all others from the dataset, also introduces a survivorship bias.

for one source of sampling bias to prevent our classifier from picking up on spurious correlations, rather than detecting malware. To this end, we construct individual datasets that exclusively contain only apps from one specific market each, instead of training on the overall, large dataset of Android apps. This ensures that the classifier learns to detect malware instead of capturing differences between the markets. For this experiment, we use the three largest markets in AndroZoo (GooglePlay, Anzhi, and AppChina) with 10,000 benign and 1,000 malicious apps each, and train DREBIN on all datasets.

The results are depicted in Table 7. The detection performance varies across the datasets, with an F1-score ranging from 0.807 to 0.879. However, if we ignore the origin of the apps and randomly sample from the complete AndroZoo dataset, we obtain the best F1-score of 0.885, indicating a clear sampling bias. This simple experiment demonstrates how controlling for a source of bias can help to better estimate the performance of a malware detector. While the example is simple and specific to Android malware, it is easily transferable to other sources and scenarios.

B Appendix: Prevalence Analysis

Here, we provide additional details on the author survey discussed in §3. Note that the supplementary material contains further information regarding the chosen papers.

Details of author survey. In addition to the discussion of the survey conducted in §3, Figure 9 provides an overview of the authors’ responses grouped by pitfall. Each bar indicates the agreement of the authors, with colors ranging from warm (strongly disagree) to cold (strongly agree).

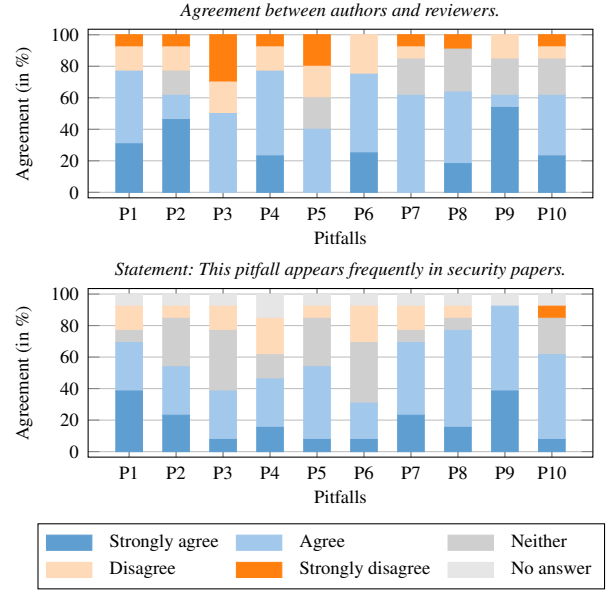


Figure 9: Survey results regarding the different pitfalls.

Data collection and ethics. Our institution does not require a formal IRB process for the survey conducted in this work. However, we contacted the ethical review board of our institution and achieved approval from its chair for conducting the survey. Moreover, we designed the survey in accordance with the General Data Protection Regulation of the EU, minimizing and anonymizing data where possible. All authors approved to a consent form that informed them about the purpose of the study, the data we collect, and included an e-mail address to contact us in case of questions.